

Algorithmique et Python : 20 exercices corrigés

maths 2nde — du geste seul au problème de contrôle

Vingt exercices, du geste seul au problème : suivre un programme ligne à ligne, reconnaître un type, lire une condition, dérouler une boucle, écrire une fonction, simuler le hasard. Un placement à 4 %, le CO₂ de l'atmosphère, un dé lancé dix mille fois, la conjecture de Syracuse, la racine de 2 cherchée pas à pas. Chaque corrigé montre la trace du programme, tour après tour. Un rappel de cours avant chaque niveau. Cherche d'abord au brouillon, puis ouvre la correction.

★ Un seul geste : exercices 1 à 8 niveau 1

Un programme par exercice : le suivre ligne à ligne en notant les valeurs.

Rappel de cours

- « $x = x + 1$ » n'est pas une égalité : c'est un ORDRE. On calcule à droite avec l'ancienne valeur, puis on range le résultat dans x.
- Les types : int (entier), float (nombre à virgule), str (texte, entre guillemets), bool (True ou False). « / » donne TOUJOURS un float ; « // » le quotient entier ; « % » le reste.
- « for i in range(1, 6) » : i prend les valeurs 1, 2, 3, 4, 5 — le 6 est EXCLU. « while condition » : on répète TANT QUE la condition est vraie.

1 Suivre ce programme ligne à ligne. Que valent a et b à la fin ?

```
a = 5
b = 3
a = a + b
b = a - b
a = a - b
```

2 Que vaut x à la fin de ce programme ?

```
x = 2
x = 3 * x + 1
x = x ** 2
```

3 Donner la valeur et le type de chaque expression :

a) « 7 » b) « 7.0 » c) « 7 / 2 » d) « 7 // 2 » e) « 7 % 2 » f) « '7' » g) « 3 < 5 »

4 Que renvoie Python pour chacune de ces expressions ?

a) « 6 / 3 » b) « '3' + '4' » c) « 3 + 4 » d) « int('3') + 4 » e) « '3' * 2 »

5 a) Qu'affiche ce programme ?

b) Et si l'on remplace la première ligne par « n = 40 » ?

```
n = 17
if n % 2 == 0:
    print('pair')
else:
    print('impair')
```

6 Un cinéma applique ce tarif. Que renvoient « prix(8) », « prix(12) », « prix(17) » et « prix(18) » ?

```
def prix(age):
    if age < 12:
        return 5
    elif age < 18:
        return 7
    else:
        return 10
```

7 Qu'affiche ce programme ? Faire la trace de i et de s.

```
s = 0
for i in range(1, 6):
    s = s + i
print(s)
```

8 Qu'affiche ce programme ? Combien de fois la boucle tourne-t-elle ?

```
n = 1
while n < 100:
    n = n * 2
print(n)
```

★★ Type devoir : exercices 9 à 16 niveau 2

Plusieurs étapes, comme au contrôle : lire une fonction, compléter un programme, en écrire un.

Rappel de cours

- Une fonction : « `def nom(x):` » puis, en retrait, les instructions, et « `return` » pour rendre le résultat.
- Une liste s'écrit entre crochets : « `[12, 8, 15]` ». « `len(L)` » donne sa longueur ; « `for n in L` » parcourt ses éléments.
- L'INDENTATION (le retrait) dit ce qui est DANS la boucle ou la condition. Elle fait partie du programme.

- 9 a) Que renvoient « `f(2)` », « `f(-1)` » et « `f(0)` » ?
b) Quelle fonction mathématique ce programme calcule-t-il ?

```
def f(x):  
    return 3 * x ** 2 - 5
```

- 10 Qu'affiche ce programme ? Que calcule-t-il ?

```
notes = [12, 8, 15, 11, 14]  
s = 0  
for n in notes:  
    s = s + n  
print(s / len(notes))
```

- 11 On place 1 000 € à 4 % par an (intérêts composés). Ce programme cherche au bout de combien d'années le capital dépasse 1 500 €.
- a) Expliquer la ligne « `c = c * 1.04` ».
- b) Qu'affiche le programme ?

```
c = 1000  
n = 0  
while c < 1500:  
    c = c * 1.04  
    n = n + 1  
print(n)
```

- 12 a) Écrire une fonction « `est_multiple(a, b)` » qui renvoie `True` si `a` est un multiple de `b`, et `False` sinon.
b) Que renvoient « `est_multiple(21, 7)` » et « `est_multiple(22, 7)` » ?
c) Quel est le type de ce qu'elle renvoie ?

- 13 Écrire un programme qui calcule et affiche la somme des carrés des entiers de 1 à 10 : $1^2 + 2^2 + \dots + 10^2$. Quelle valeur affiche-t-il ?

14 Voici l'algorithme de Syracuse : si n est pair, on le divise par 2 ; sinon, on le multiplie par 3 et on ajoute 1. On recommence jusqu'à obtenir 1.

- a) Qu'affiche le programme ?
- b) Que compte la variable `etapes` ?

```
n = 6
etapes = 0
while n != 1:
    if n % 2 == 0:
        n = n // 2
    else:
        n = 3 * n + 1
    etapes = etapes + 1
print(etapes)
```

15 Ce programme doit afficher l'âge de l'utilisateur l'an prochain. Il s'arrête sur une erreur.

- a) Pourquoi ?
- b) Corriger la première ligne.

```
age = input('Âge ? ')
print(age + 1)
```

16 a) Que renvoient « `somme(4)` » et « `somme(100)` » ?

- b) Comparer avec la formule $\frac{n(n+1)}{2}$.

```
def somme(n):
    s = 0
    for k in range(n + 1):
        s = s + k
    return s
```

★★★ Problèmes : exercices 17 à 20 niveau 3

Des situations réelles : on lit le programme, on le fait tourner à la main, on interprète ce qu'il affiche.

Rappel de cours

- « `from random import randint` » puis « `randint(1, 6)` » : un entier au hasard entre 1 et 6, bornes COMPRISES.
- Simuler, c'est répéter une expérience aléatoire un grand nombre de fois et compter : la fréquence obtenue s'approche de la probabilité.
- Un programme ne prouve rien d'un modèle : il calcule ce que le modèle PRÉVOIT.

17 Le dé lancé dix mille fois

- a) Que calcule la fonction « lancers(n) » ?
- b) Quelle valeur s'attend-on à trouver pour « lancers(10000) » ?
- c) Deux élèves lancent « lancers(10) » : l'un obtient 0.4, l'autre 0.0. L'un d'eux s'est-il trompé ?

```
from random import randint

def lancers(n):
    six = 0
    for i in range(n):
        if randint(1, 6) == 6:
            six = six + 1
    return six / n
```

18 Le CO₂ de l'atmosphère

À l'observatoire de Mauna Loa (Hawaï), la concentration moyenne de CO₂ dans l'air était d'environ 424 ppm (parties par million) en 2024. On suppose qu'elle augmente de 2,5 ppm par an.

- a) En quelle année le programme prévoit-il le passage des 450 ppm ?
- b) Quel est le type de co2 à la fin ? Pourquoi ?
- c) Que faut-il changer pour une hausse de 3 ppm par an ? Quelle année obtient-on alors ?

```
co2 = 424
annee = 2024
while co2 < 450:
    co2 = co2 + 2.5
    annee = annee + 1
print(annee)
```

19 Deux dés : le 7 ou le 12 ?

On lance deux dés et on fait la somme. Ce programme compte, sur 36 000 lancers, combien de fois on obtient 7 et combien de fois 12.

- a) Compléter les deux lignes « if ».
- b) Quelles fréquences s'attend-on à trouver ? Justifier avec les 36 issues possibles.
- c) Un joueur parie sur le 12 « parce que c'est le plus gros ». A-t-il raison ?

```
from random import randint
sept = 0
douze = 0
for i in range(36000):
    d1 = randint(1, 6)
    d2 = randint(1, 6)
    s = d1 + d2
    if ...:
        sept = sept + 1
    if ...:
        douze = douze + 1
print(sept, douze)
```

20 La racine de 2, pas à pas

On cherche $\sqrt{2}$ sans la touche racine : on part de 1 et on avance de 0,1 tant que le carré reste sous 2.

- a) Faire la trace de x. Quel encadrement de $\sqrt{2}$ obtient-on ?
- b) Python affiche en fait « 1.5000000000000004 ». Pourquoi pas « 1.5 » ?
- c) Que faut-il changer pour avoir un encadrement à 0,01 près ? Python affiche alors « 1.4200000000000004 » : quel encadrement en déduit-on ?

```
x = 1
while x * x < 2:
    x = x + 0.1
print(x)
```

Corrigés : algorithmique et python, 20 exercices

Ces pages se détachent de la feuille d'exercices. Chaque étape est écrite, avec son pourquoi.

1. On note les valeurs après chaque ligne. À droite du signe « = », on utilise les valeurs du moment.

Ligne 3 : a reçoit $5 + 3 = 8$. Ligne 4 : b reçoit $8 - 3 = 5$. Ligne 5 : a reçoit $8 - 5 = 3$.

À la fin, a vaut 3 et b vaut 5 : le programme a ÉCHANGÉ les deux valeurs.

⊖ Le piège : lire « $a = a + b$ » comme une équation, et conclure que b vaut 0. En Python, « = » range une valeur, il n'affirme rien.

ligne	a	b
1	5	-
2	5	3
3	8	3
4	8	5
5	3	5

2. Ligne 2 : on calcule $3 \times 2 + 1 = 7$ avec l'ANCIENNE valeur de x, puis x vaut 7.

Ligne 3 : « ** » est la puissance : x reçoit $7^2 = 49$.

À la fin, x vaut 49.

⊖ Le piège : écrire le carré avec l'accent circonflexe, comme sur la calculatrice. En Python, le carré s'écrit « $x ** 2$ » ; l'accent circonflexe y fait une tout autre opération.

ligne	x
1	2
2	7
3	49

3. a) 7, un entier : int.

b) 7,0, un nombre à virgule : float (le point est la virgule de Python).

c) 3,5 : float. La division « / » donne toujours un float.

d) 3 : int. « // » donne le quotient entier ($7 = 2 \times 3 + 1$).

e) 1 : int. « % » donne le reste de la division.

f) le texte « 7 », entre guillemets : str. Ce n'est plus un nombre.

g) True : bool. Une comparaison vaut vrai ou faux.

⊖ Le piège au f) : croire que '7' et 7 sont la même chose. Les guillemets en font un TEXTE.

expression	valeur	type
7	7	int
7.0	7.0	float
7 / 2	3.5	float
7 // 2	3	int
7 % 2	1	int
'7'	'7'	str
3 < 5	True	bool

4. a) « 2.0 », et non « 2 » : « / » donne toujours un float, même quand la division tombe juste.

b) « '34' » : entre deux textes, « + » les COLLE bout à bout.

c) « 7 » : entre deux nombres, « + » additionne.

d) « 7 » : int('3') transforme le texte '3' en nombre 3, puis on ajoute 4.

e) « '33' » : un texte multiplié par 2 est répété deux fois.

★ Le même signe « + » ne fait pas la même chose selon le TYPE de ce qu'il relie.

expression	résultat
6 / 3	2.0
'3' + '4'	'34'
3 + 4	7
int('3') + 4	7
'3' * 2	'33'

5. a) « $n \% 2$ » est le reste de la division de n par 2 : $17 = 2 \times 8 + 1$, le reste vaut 1. La condition « $1 == 0$ » est fausse : on passe dans le « else », le programme affiche « impair ».

b) $40 = 2 \times 20 + 0$, le reste vaut 0 : la condition est vraie, il affiche « pair ».

⊖ Le piège : écrire « if $n \% 2 = 0$ ». Un seul « = » RANGE une valeur ; pour COMPARER, il en faut deux : « == ».

n	$n \% 2$	$n \% 2 == 0$	affiche
17	1	False	impair
40	0	True	pair

6. On teste les conditions DANS L'ORDRE, et on s'arrête à la première vraie.

prix(8) : $8 < 12$ est vrai, on renvoie 5.

prix(12) : $12 < 12$ est FAUX ; $12 < 18$ est vrai, on renvoie 7.

prix(17) : $17 < 12$ faux, $17 < 18$ vrai : 7.

prix(18) : les deux sont faux, on arrive au « else » : 10.

⊖ Le piège : prix(12) = 5. Avec « < », 12 n'est pas strictement inférieur à 12 : à 12 ans, on paie déjà 7 €.

âge	age < 12	age < 18	prix
8	True	-	5
12	False	True	7
17	False	True	7
18	False	False	10

7. « range(1, 6) » donne $i = 1, 2, 3, 4, 5$: le 6 est exclu.

À chaque tour, on ajoute i à s : $0 + 1 = 1$, puis 3, 6, 10, 15.

Le programme affiche 15 : c'est $1 + 2 + 3 + 4 + 5$.

⊖ Le piège : faire aussi un tour avec $i = 6$, et trouver 21. La borne de droite d'un range n'est jamais atteinte.

tour	i	s
1	1	1
2	2	3
3	3	6
4	4	10
5	5	15

8. On répète TANT QUE n est inférieur à 100, et on teste AVANT chaque tour.

n vaut successivement 1, 2, 4, 8, 16, 32, 64 : à chaque fois, $n < 100$, on continue.

Avec $n = 64$, on fait encore un tour : n devient 128. Maintenant $128 < 100$ est faux : on sort.

La boucle a tourné 7 fois, et le programme affiche 128.

⊖ Le piège : répondre 64, « le dernier nombre sous 100 ». La boucle s'arrête quand la condition devient FAUSSE, donc après l'avoir dépassé.

tour	n avant	n < 100 ?	n après
1	1	oui	2
2	2	oui	4
3	4	oui	8
4	8	oui	16
5	16	oui	32
6	32	oui	64
7	64	oui	128

9. a) On remplace x par la valeur donnée. « ** » passe avant « * », qui passe avant « - » : les priorités des maths.

$$f(2) = 3 \times 2^2 - 5 = 12 - 5 = 7.$$

$$f(-1) = 3 \times (-1)^2 - 5 = 3 - 5 = -2.$$

$$f(0) = 0 - 5 = -5.$$

b) C'est la fonction $x \mapsto 3x^2 - 5$.

⊖ Le piège au a) : taper « -1 ** 2 » dans la console. Python calcule d'abord la puissance : il obtient $-(1^2) = -1$. Quand on appelle f(-1), la valeur -1 remplace x tout entier, comme si on écrivait $(-1)^2$.

appel	calcul	renvoie
f(2)	3 * 4 - 5	7
f(-1)	3 * 1 - 5	-2
f(0)	3 * 0 - 5	-5

10. La boucle parcourt la liste : n prend les valeurs 12, 8, 15, 11, 14, et s les additionne : 12, 20, 35, 46, 60.

« len(notes) » vaut 5, le nombre de notes.

Le programme affiche « 12.0 » : c'est la MOYENNE des notes, $\frac{60}{5} = 12$.

★ « 12.0 » et non « 12 » : la division « / » donne un float, même quand elle tombe juste.

tour	n	s
1	12	12
2	8	20
3	15	35
4	11	46
5	14	60

11. a) Augmenter de 4 %, c'est multiplier par 1,04. Chaque tour de boucle fait passer une année.

b) On déroule, au centime : 1 040 ; 1 081,60 ; 1 124,86 ... ; au bout de 10 ans, 1 480,24 € : encore sous 1 500, on continue. Au bout de 11 ans, 1 539,45 € : on sort.

Le programme affiche 11 : il faut 11 années.

★ C'est un algorithme de SEUIL : on ne sait pas d'avance combien de tours il faut, d'où le « while ».

⊖ Le piège : répondre 12,5 ans, « 500 € à 40 € par an ». Les intérêts s'ajoutent aux intérêts : le capital grandit plus vite.

n (années)	c (€)
0	1 000
1	1 040
2	1 081,60

n (années)	c (€)
9	1 423,31
10	1 480,24
11	1 539,45

12. a) a est un multiple de b quand le reste de la division de a par b est nul :

« def est_multiple(a, b): » puis, en retrait, « return a % b == 0 ».

b) $21 = 7 \times 3 + 0$: le reste est 0, elle renvoie True. $22 = 7 \times 3 + 1$: le reste est 1, elle renvoie False.

c) Une comparaison vaut True ou False : la fonction renvoie un bool.

★ Pas besoin de « if » : la comparaison « a % b == 0 » EST déjà le résultat.

```
def est_multiple(a, b):
    return a % b == 0

est_multiple(21, 7) # True
est_multiple(22, 7) # False
```

13. On garde le schéma de l'exercice 7 : une variable s à 0, puis une boucle qui ajoute chaque carré.

Les entiers de 1 à 10 : « range(1, 11) », car la borne de droite est exclue.

À chaque tour, on ajoute « k ** 2 ».

Le programme affiche 385.

★ Vérification : $1 + 4 + 9 + 16 + 25 + 36 + 49 + 64 + 81 + 100 = 385$.

⊖ Le piège : « range(1, 10) », qui s'arrête à 9 et oublie $10^2 = 100$.

```
s = 0
for k in range(1, 11):
    s = s + k ** 2
print(s)
```

14. a) On déroule : 6 est pair → 3 ; 3 est impair → 10 ; puis 5, 16, 8, 4, 2, 1.

On a obtenu 1 : « n != 1 » (n différent de 1) est faux, on sort. Le programme affiche 8.

b) etapes compte le nombre de transformations pour arriver à 1 : ici 8.

★ Personne n'a jamais trouvé de nombre qui n'arrive pas à 1... mais personne n'a démontré que c'était toujours vrai : c'est la CONJECTURE de Syracuse, toujours ouverte.

⊖ Le piège : « n / 2 » au lieu de « n // 2 ». On obtiendrait des float, « 3.0 », « 5.0 »...

étape	n
0	6
1	3
2	10
3	5
4	16
5	8
6	4
7	2
8	1

15. a) « input » renvoie TOUJOURS un texte (str), même si l'on tape des chiffres : age vaut par exemple '15', entre guillemets.

« age + 1 » demande d'ajouter un texte et un nombre : Python refuse (TypeError).

b) On convertit le texte en entier dès la saisie : « age = int(input('Âge ? ')) ». Avec 15, le programme affiche 16.

★ Le type d'une variable n'est pas écrit dans le programme, mais il décide de ce qu'on peut en faire.

```
age = int(input('Âge ? '))
print(age + 1)
```

16. a) « range(n + 1) » donne $k = 0, 1, \dots, n$. La fonction additionne donc les entiers de 0 à n .

somme(4) : $0 + 1 + 2 + 3 + 4 = 10$. somme(100) : 5 050.

b) $\frac{4 \times 5}{2} = 10 \checkmark$ et $\frac{100 \times 101}{2} = 5\,050 \checkmark$.

★ La boucle fait 101 tours pour somme(100) ; la formule, un seul calcul. Le programme sert à VÉRIFIER une formule sur beaucoup de cas avant de la démontrer.

⊖ Le piège : « range(n) », qui s'arrête à $n - 1$ et oublie le dernier terme.

k	s
0	0
1	1
2	3
3	6
4	10

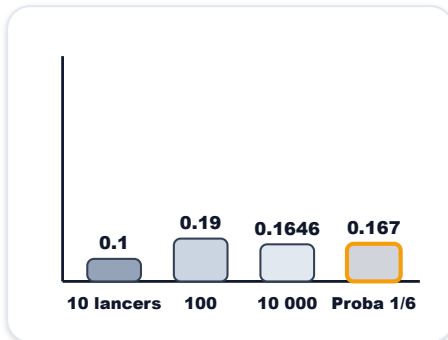
17. Le dé lancé dix mille fois

a) La boucle lance n fois un dé ; à chaque « 6 », on ajoute 1 au compteur six. À la fin, on divise par n : la fonction renvoie la FRÉQUENCE des six sur n lancers.

b) La probabilité d'un six est $\frac{1}{6} \approx 0,167$. Sur 10 000 lancers, la fréquence est très proche de 0,167 : c'est la loi des grands nombres.

c) Personne ne s'est trompé. Sur 10 lancers, obtenir 4 six ou aucun est tout à fait possible : un petit échantillon FLUCTUE beaucoup.

★ Le diagramme montre un vrai essai : 0,1 sur 10 lancers, 0,19 sur 100, 0,1646 sur 10 000. Plus n est grand, plus la fréquence se resserre autour de $\frac{1}{6}$: c'est pour cela qu'on simule en grand.



18. Le CO₂ de l'atmosphère

a) Il faut monter de $450 - 424 = 26$ ppm. Au bout de 10 ans, $424 + 25 = 449$: encore sous 450. Au bout de 11 ans, 451,5 : on sort.

Le programme affiche 2035.

b) co2 valait l'entier 424, mais on lui ajoute 2,5 : il devient un float, « 451.5 ».

c) On remplace « 2.5 » par « 3 ». Il faut alors 9 ans ($424 + 27 = 451$) : le programme affiche 2033.

★ Le modèle ne dit rien de l'avenir réel : il dit ce qui arrive SI la hausse reste constante. En 2024, elle a d'ailleurs été plus forte que 2,5 ppm.

⊖ Le piège au a) : diviser, $\frac{26}{2,5} = 10,4$, et répondre 2034. On compte des années entières, et 10 ans ne suffisent pas.

annee	co2
2024	424

annee	co2
2025	426.5
2033	446.5
2034	449.0
2035	451.5

19. Deux dés : le 7 ou le 12 ?

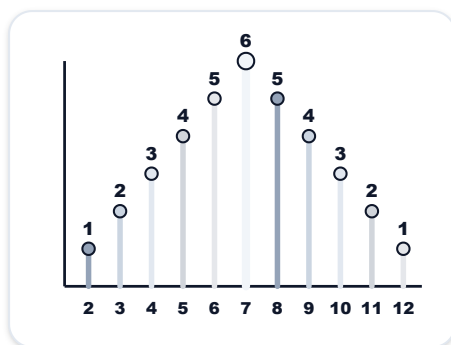
a) « if s == 7: » et « if s == 12: » (deux « = » pour comparer).

b) Deux dés donnent $6 \times 6 = 36$ issues équiprobables. La somme 7 en compte 6 : (1 ; 6), (2 ; 5), (3 ; 4), (4 ; 3), (5 ; 2), (6 ; 1). La somme 12 une seule : (6 ; 6).

On attend une fréquence proche de $\frac{6}{36} = \frac{1}{6}$ pour 7, soit environ 6 000 fois ; et proche de $\frac{1}{36}$ pour 12, soit environ 1 000 fois.

c) Non : le 7 sort environ SIX fois plus souvent. Toutes les sommes ne sont pas équiprobables, même si chaque dé l'est.

❌ Le piège au b) : croire les onze sommes, de 2 à 12, également probables. Ce sont les 36 COUPLES qui le sont.



20. La racine de 2, pas à pas

a) x vaut 1, puis 1,1 ; 1,2 ; 1,3 ; 1,4 : leurs carrés, 1,21 ; 1,44 ; 1,69 ; 1,96, sont sous 2. Avec 1,5, le carré vaut 2,25 : on sort. $1,4^2 < 2 < 1,5^2$, donc $1,4 < \sqrt{2} < 1,5$.

b) L'ordinateur calcule en BINAIRE, où 0,1 ne s'écrit pas exactement (comme $\frac{1}{3}$ en décimal). Chaque ajout garde une minuscule erreur, et elles s'accumulent : 1,5000000000000004 au lieu de 1,5.

c) On remplace « 0.1 » par « 0.01 ». Le programme s'arrête au premier x dont le carré dépasse 2, soit 1,42 (à l'erreur près) : $1,41 < \sqrt{2} < 1,42$.

★ C'est un algorithme de BALAYAGE : plus le pas est petit, plus l'encadrement est fin — et plus la boucle tourne.

❌ Le piège au a) : répondre « $\sqrt{2} \approx 1,5$ ». Le programme donne la PREMIÈRE valeur trop grande : $\sqrt{2}$ est juste avant.

x	x * x	< 2 ?
1	1	oui
1.1	1.21	oui
1.2	1.44	oui
1.3	1.69	oui
1.4	1.96	oui
1.5	2.25	non